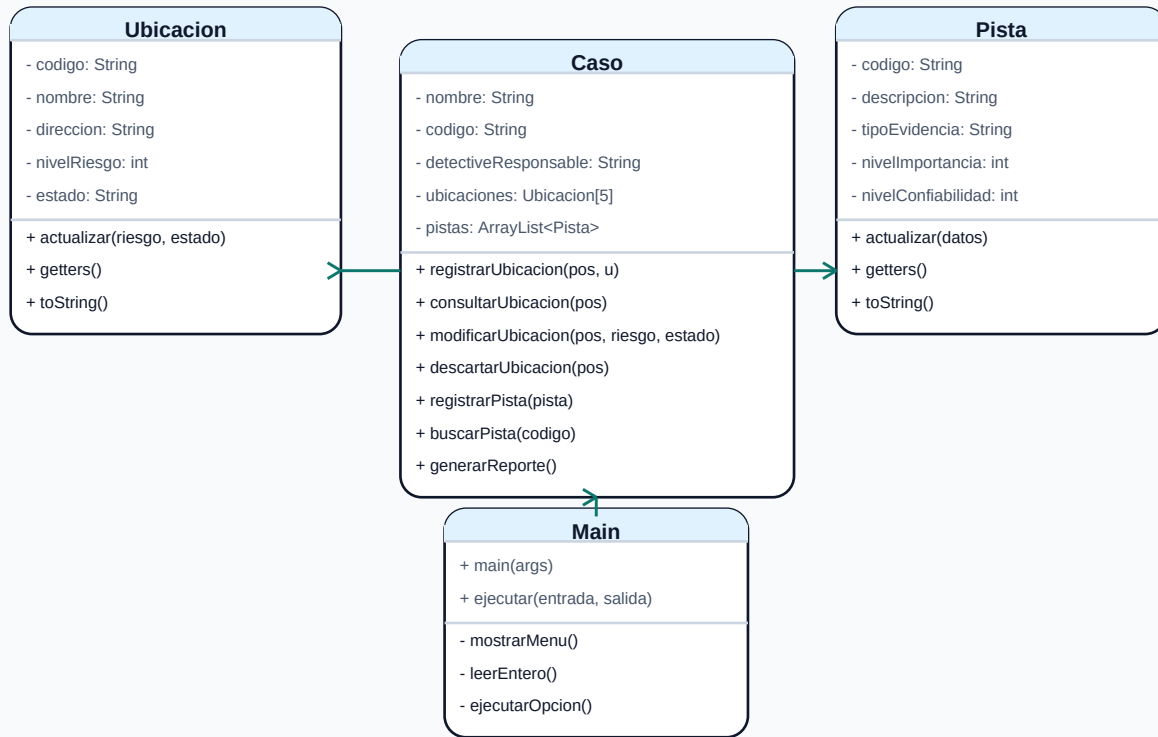


LABORATORIO 2

Arreglos y excepciones



Main usa Caso; Caso compone el arreglo fijo y el ArrayList.

AGENCIA DE DETECTIVES - CASO MISTERIOSO

Documento de análisis, diseño y decisiones de implementación

Curso	Programación Orientada a Objetos
Tema	Arreglos de objetos, ArrayList y excepciones
Programa	Java de consola
Estructuras principales	Ubicacion[5] y ArrayList

1. Objetivo y alcance

El sistema organiza la información recopilada por una agencia privada de detectives. El caso administra una cantidad fija de ubicaciones mediante un arreglo básico de cinco objetos y una cantidad dinámica de pistas mediante un ArrayList. La solución prioriza encapsulamiento, validación y recuperación ante errores de entrada.

Criterios de alcance

- El programa es una aplicación de consola y no depende de bibliotecas externas.
- Las posiciones vacías del arreglo se conservan como null y se validan antes de usar sus métodos.
- Las pistas se consultan y modifican recorriendo el ArrayList por código.
- El menú mantiene la ejecución después de excepciones recuperables.

2. Requisitos funcionales

ID	Requisito
RF-01	Solicitar nombre del caso, código de identificación y detective responsable.
RF-02	Administrar un máximo de cinco ubicaciones en un arreglo básico de objetos.
RF-03	Almacenar código, nombre, dirección o descripción, nivel de riesgo y estado de cada ubicación.
RF-04	Validar el nivel de riesgo entre 1 y 10 al crear o modificar una ubicación.
RF-05	Registrar una ubicación en la posición indicada, verificando límites, disponibilidad y datos.
RF-06	Reconocer y mantener null en las posiciones del arreglo todavía vacías.
RF-07	Consultar todas las ubicaciones sin mostrar posiciones null como existentes.
RF-08	Consultar una ubicación específica y reportar posiciones inválidas o vacías.
RF-09	Modificar el nivel de riesgo y el estado de una ubicación existente.
RF-10	Descartar una ubicación y devolver su posición a null.
RF-11	Almacenar una cantidad desconocida de pistas directamente en ArrayList.
RF-12	Almacenar código, descripción, tipo de evidencia, importancia y confiabilidad de cada pista.
RF-13	Validar importancia entre 1 y 10 y confiabilidad entre 0 y 100.
RF-14	Registrar pistas y rechazar códigos repetidos.
RF-15	Consultar todas las pistas y manejar correctamente una colección vacía.
RF-16	Buscar una pista mediante su código recorriendo el ArrayList.
RF-17	Modificar una pista existente con sus validaciones.
RF-18	Eliminar una pista localizada por código.
RF-19	Calcular cantidades, máximos y promedio solicitados para ubicaciones y pistas.
RF-20	Manejar InputMismatchException al solicitar números con Scanner.
RF-21	Usar try-catch y finally para mantener el menú activo después de errores.
RF-22	Incluir un driver program con un menú de 13 opciones.

3. Análisis de clases

Clase	Responsabilidad dentro del sistema
Caso	Representar la investigación activa y coordinar el arreglo de ubicaciones y el ArrayList de pistas.
Ubicacion	Representar un lugar investigado y proteger la regla del nivel de riesgo.
Pista	Representar una evidencia y proteger las reglas de importancia y confiabilidad.
Main	Ser el driver program, mostrar el menú, leer entradas y manejar excepciones de interacción.

3.1 Atributos

Clase	Atributo	Tipo	Propósito
Caso	nombre, codigo, detectiveResponsable	String	Identificar la investigación y su responsable.
Caso	ubicaciones	Ubicacion[]	Arreglo fijo de cinco posiciones; sus espacios vacíos contienen null.
Caso	pistas	ArrayList	Colección dinámica obligatoria para evidencias encontradas.
Ubicacion	codigo, nombre, direccion, estado	String	Describir e identificar el lugar.
Ubicacion	nivelRiesgo	int	Representar el riesgo validado de 1 a 10.
Pista	codigo, descripcion, tipoEvidencia	String	Identificar y describir la evidencia.
Pista	nivelImportancia	int	Valorar la evidencia de 1 a 10.
Pista	nivelConfiabilidad	int	Valorar la confiabilidad de 0 a 100.

3.2 Encapsulamiento

Los atributos son privados. Las clases de dominio exponen getters para consultar información y métodos de operación para modificar únicamente los valores permitidos. Los constructores y métodos de actualización comparten la responsabilidad de validar antes de cambiar el estado.

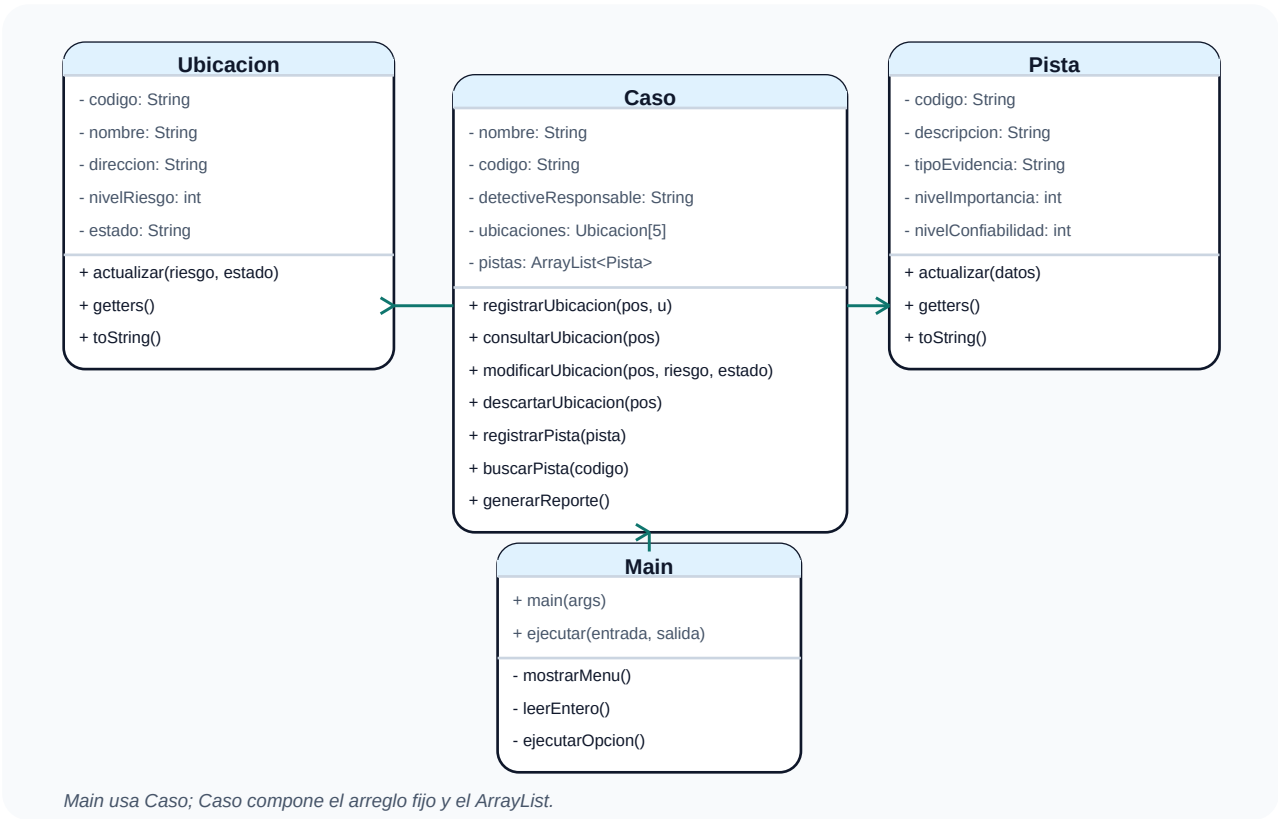
4. Métodos y excepciones

Clase	Método	Resultado y validaciones
Caso	registrarUbicacion(posicion, ubicacion)	Valida límites, null y disponibilidad antes de asignar el objeto.
Caso	consultarUbicaciones() / consultarUbicacion(posicion)	Recorre el arreglo y evita usar métodos sobre null.
Caso	modificarUbicacion / descartarUbicacion	Rechaza posiciones inválidas o vacías.
Caso	registrarPista / buscarPista	Recorre ArrayList y evita códigos duplicados.
Caso	modificarPista / eliminarPista	Busca por código y rechaza pistas inexistentes.
Caso	generarReporte()	Calcula cantidades, máximos y promedio sin operar sobre una colección vacía.
Ubicacion	actualizar(riesgo, estado)	Valida riesgo 1-10 y estado no vacío.
Pista	actualizar(datos)	Valida importancia 1-10 y confiabilidad 0-100.
Main	leerEntero()	Captura InputMismatchException, limpia el token y vuelve a solicitar.
Main	ejecutarOpcion()	Captura IllegalArgumentException y deja disponible el menú.

El bloque finally se ejecuta al terminar cada operación del menú para informar que la operación concluyó, incluso si la validación produjo una excepción. Así se deja claro al usuario que puede continuar con otra opción.

5. Diseño UML

El diagrama muestra el driver Main, la composición de Caso con sus dos estructuras de almacenamiento y las entidades que representan los datos. Los signos + y - representan visibilidad pública y privada, respectivamente.



Relación	Justificación
Main -> Caso	Main crea el caso activo y delega las operaciones del menú.
Caso -> Ubicacion[]	Un caso tiene exactamente cinco espacios para ubicaciones.
Caso -> ArrayList	Un caso administra una cantidad dinámica de pistas.

6. Menú del programa

Opción	Operación
1	Nuevo caso: reemplaza el caso actual y reinicia sus estructuras.
2-6	Registrar, consultar, modificar y descartar ubicaciones.
7-11	Registrar, consultar, buscar, modificar y eliminar pistas.
12	Mostrar reporte de investigación con todos los cálculos pedidos.
13	Salir de la aplicación.

7. Verificación propuesta

- Compilar todas las clases con el JDK sin dependencias externas.

- Crear un caso y registrar una ubicación en una posición disponible.
- Intentar usar una posición inválida y confirmar que el programa continúa.
- Ingresar texto donde se solicita un número y confirmar la recuperación del Scanner.
- Registrar dos pistas, verificar búsqueda, modificación, eliminación y reporte.
- Confirmar que el reporte vacío no intenta calcular máximos ni promedio.

Conclusión

La solución separa los datos de la interacción, conserva las estructuras exigidas por la guía y concentra las reglas de validación en las clases que protegen cada objeto. Esto permite que el menú sea sencillo de seguir y que las operaciones se puedan comprobar de forma independiente.